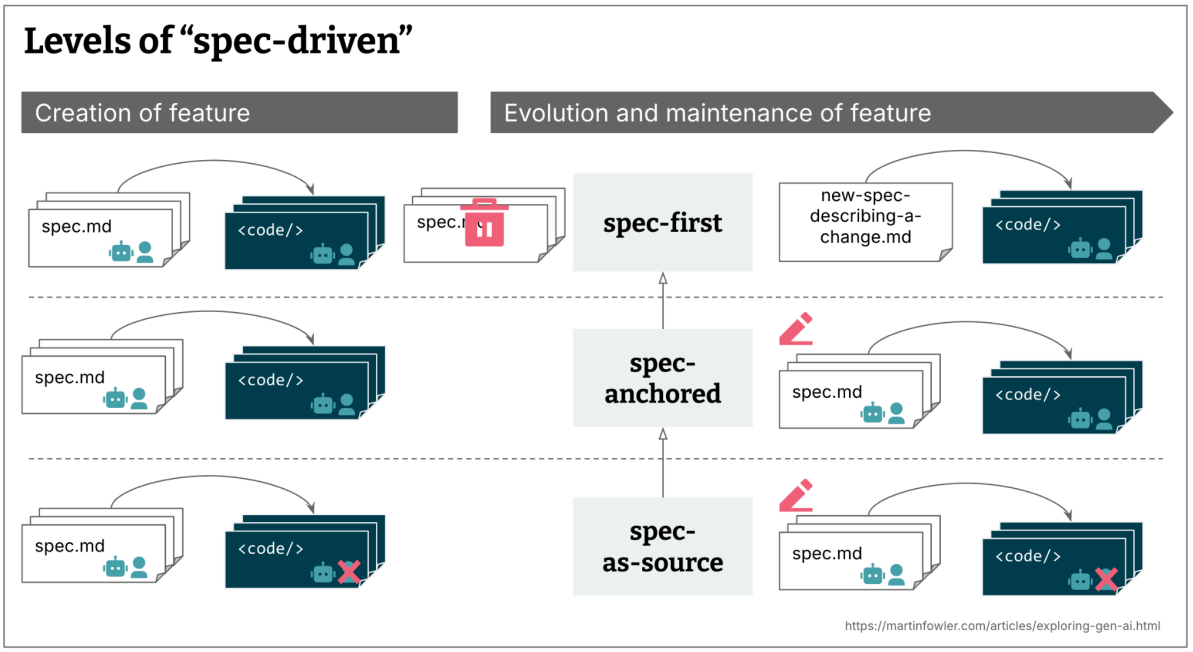
Specification-Driven Development
Spezifikation-getriebene Entwicklung

Keynote von Birgitta Boeckeler an der OOP-Konferenz
https://speakerdeck.com/birgitta410/state-of-play-ki-unterstutzte-programmierung-oop-2026-keynote



https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html

Aufgabenblatt: Experimentieren mit SDD

Aufgabenstellung
Leitfrage: Lässt sich mit einer Anforderungsspezifikation eine Software generieren?
Lernziel: SDD verstehen und einschätzen können
Arbeit in Kleingruppen (3-4 Personen)

Anwendungsbeispiel: Klassenchat, Spezifikations-verankert

Tooling
Spezifikation: ChatGPT.com (darf auch Alternative sein)
Modelle: PlantUML.com
Repository: GitHub.com
Sprache: Python.org
Web-Applikationsframework: Flask.palletsprojects.com/en/stable/
Webserver: gunicorn.org
Betrieb: Render.com

We would like to write a classroom messenger. - As simple and basic as possible. - Uses a simple, basic, commodity cloud service. - Realised with simple Python code. Software Engineering Methodology: - We follow specification-driven development - We use render.com

Arbeitsziele und -Resultate
A1) Die Anforderungen an die Applikation definieren, KI-assistiert
A2) Komplementäre Spezifikationen: Architektur, Deployment.
A3) Laufende Applikation (TRL4-Prototyp)
A4) Reflexion zum Einsatz von SDD, auch in eigener Organisation: SWOT

Vorgehen: Total ca. 2x 90 Minuten
- DDD: zentrale Konzepte für die Nutzenden
-

Schritt 1: Konzepte als Klassen ausgedrückt (Domain-Driven Development, DDD)

Brainstorming: Welches sind die für Nutzende zentralen Konzepte?
Resultat: Klassendiagramm mit Klassen, Attribute, Beziehungen
Qualitätssicherung: Peer-Review

Schritt 2: Funktionale Anforderungen in Features gruppiert

Brainstorming: Welches sind die für ein MVP notwendigen Features und Anforderungen?
Resultat: Katalog von Features (Zweck, Anforderungen, Umsetzungsidee)

Qualitätssicherung: Peer-Review

Schritt 3: Interaktionsszenarien als Use Cases

Use-case Methodik und Ableitung von Anforderungen gemäss UML-Block im REQ-Skript.
Resultat 1: Use Cases mit Interaktionsszenarien.
Resultat 2: Komplette Liste mit funktionalen Anforderungen.
Qualitätssicherung: Peer-Review

Schritt 4: Requirements Quality Check

Lasse KI Konsistenz, Eindeutigkeit, Notwendigkeit und Komplettheit für ein MVP beurteilen.
Verbessere und finalisiere die Spezifikation manuell.

Schritt 5: Minimales UI-Design

Generiere ein passendes, minimales UI
Qualitätseigenschaften: Komplette Abdeckung der Anforderungen, minimal, NN/g Usability
Resultat: Lauffähiges HTML mit simuliertem Verhalten
Qualitätssicherung: Nutzertest

Schritt 6: Code-Generierung und Deployment (TRL4-Prototyp)

Die folgende Architekturbeschreibung wurde mit Perplexity.ai generiert

- Prompt 1:
  On which services can I host a Python application, as easy as possible? I need a database and a web UI. My app is a simple web-based messenger.
- Prompt 2:
  Please explain how I would need to proceed to have my app deployed and running on render.com.
- Prompt 3:
  Create a very short human and AI-readable architecture documentation, including identification of components, required documents and folders, technology stack, and deployment.
- Prompt 4:
  Simplify: No database; we keep the messages in the Python app's memory (not persistent).
  Simplify: Have only one single endpoint.
  Please adapt so that I can easily deploy on render.com.

```
# Architecture

## Purpose
```

Simple web-based messenger with one page and one endpoint.

Components
- Browser UI
- Flask app
- In-memory message store

Endpoint
- `GET /` show page
- `POST /` send message

Stack
- Python
- Flask
- Gunicorn
- HTML/CSS
- Render Web Service

Files
- `app.py`
- `templates/index.html`
- `static/style.css`
- `requirements.txt`
- `render.yaml`
- `README.md`

Deployment
Render Web Service with:
- Build command: `pip install -r requirements.txt`
- Start command: `gunicorn app:app`

Limitation
Messages are lost on restart or redeploy.

Generiere die für das Deployment der Applikation notwendige Dateien.
Resultat: ZIP für GitHub
Constraints: Architektur für Flask-basierte Applikation auf Render.

Lade die Dateien ins GitHub, Verknüpfe Render mit GitHub, Builde und Starte.
Qualitätssicherung: Nutzertest

Reflexion: SWOT

Reflektiere das Erlebte aus der Perspektive eurer Organisationen.

Stärken	**Möglichkeiten**
Schwächen	**Gefahren**